

# Risk Engine & Multi-Threat Arbitration Specification

Technical Guide to Stopping Distance Physics, TTC Kinematics, and Safety Control Arbitration

| METRIC / SCOPE                               | SPECIFICATION / TARGET                    |
|----------------------------------------------|-------------------------------------------|
| Subsystem: Risk & Decision Engine            | Module Path: core_safar_logic/risk_engine |
| Critical TTC Threshold: $\leq 1.8s$          | Hysteresis Hold Timer: 0.35s              |
| Arbitration Hierarchy: Life Safety > Craters | Test Pass Rate: 100% (18 / 18 Unit Tests) |

## 1. Subsystem Architecture & Threat Arbitration

The Risk and Decision Subsystem acts as the central cognitive brain of SAFAR. It continuously ingests tracked obstacles and road surface hazards, computes physical stopping envelopes, evaluates safe Time-To-Collision (TTC), and arbitrates among multiple simultaneous hazards onto a single authoritative vehicle actuation command.

## 2. Decision State Machine & Multi-Threat Hierarchy

- **State Machine:** `PASSIVE`  $\rightarrow$  `ASSESSING`  $\rightarrow$  `THREAT_CONFIRMED`  $\rightarrow$  `INTERVENTION`  $\rightarrow$  `CLEARED`.
- **Active Hold Timer (0.35s):** Prevents single-frame de-escalation chatter.
- **Life Safety Precedence:** Forward vehicle and pedestrian collisions ALWAYS dominate road surface potholes. The system will never release brakes or swerve toward vulnerable road users to preserve suspension on a crater.

## 3. Source Code Files Breakdown

| FILE NAME                          | RESPONSIBILITY & ARCHITECTURAL WORK                                                                                                                          |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| risk_engine/risk_engine.py         | Implements <code>RiskEngine</code> and <code>HazardRiskEngine</code> . Evaluates dynamic stopping distance envelopes across speeds from 20 km/h to 120 km/h. |
| risk_engine/ttc.py                 | Implements <code>calculate_ttc_s()</code> . Calculates safe Time-To-Collision. Rejects divergent objects ( $v_{\text{closing}} \leq 0$ ).                    |
| risk_engine/lead.py                | Implements <code>select_lead()</code> . Identifies the primary persistent lead vehicle inside the ego corridor.                                              |
| risk_engine/models.py              | Declares data models: <code>HazardCandidate</code> , <code>HazardRiskAssessment</code> , <code>VehicleSnapshot</code> , <code>DecisionState</code> .         |
| decision_engine/arbitration.py     | Implements <code>ThreatArbiter</code> . Resolves conflicts between obstacle emergency braking and pothole slowdowns.                                         |
| decision_engine/decision_engine.py | Implements <code>DecisionEngine</code> . Manages 5-stage state transitions and hysteresis de-escalation.                                                     |
| decision_engine/policy.py          | Implements <code>HazardPolicy</code> . Defines graduated deceleration thresholds for urban vs. highway driving.                                              |

## 4. Line-by-Line Code Walkthrough: Multi-Threat Arbitration

```
class ThreatArbiter:
    def arbitrate_with_road_hazard(self, obstacle_dec: Decision, pothole_plan: Any) -> Decision:
        if pothole_plan is None: return obstacle_dec
        # 1. Forward collision emergency brake ALWAYS dominates
        if obstacle_dec.action == ActionType.EMERGENCY_BRAKE:
            return obstacle_dec
        # 2. Severe crater triggers emergency braking if obstacle path is clear
        if getattr(pothole_plan, 'hazard_class', 0) == 3 and pothole_plan.brake_command > 0.6:
            return Decision(ActionType.EMERGENCY_BRAKE, 0.0, max(obstacle_dec.brake, pothole_plan.brake_command), 0.0, f'Pothole: {pothole_plan}')
        # 3. Deceleration Blending: Take highest required brake effort
        eff_brake = max(obstacle_dec.brake, pothole_plan.brake_command)
        eff_throttle = 0.0 if eff_brake > 0.15 else obstacle_dec.throttle
        return Decision(ActionType.SLOWDOWN, min(obstacle_dec.target_speed_mps, pothole_plan.target_speed_mps), eff_brake, eff_throttle, pothole_plan)
```

**Code Explanation:** Lines 4-6 establish the fundamental safety invariant: dynamic collision avoidance takes precedence over road hazard avoidance. Lines 7-9 allow severe craters to execute full emergency braking if the obstacle path is clear. Lines 11-13 blend deceleration and suppress throttle if brake effort exceeds 15%.